

diffusion

Repo guide

Statement of the problem

Given examples of stochastic trajectories $\mathbf{x}$ —and optionally some conditioning information $\mathbf{c}$ —learn the probability distribution $p(\mathbf{x} \mid \mathbf{c})$ of trajectories well enough to generate new statistically equivalent realisations.

```
1 import torch
2 from torch.utils.data import TensorDataset
3
4 from diffusionsde import DDPM, DiffusionConfig
5
6 config = DiffusionConfig(
7     trajectory_length=50,
8     n_output_channels=2,
9     n_conditioning_channels=1,
10    hidden_dim=64,
11    n_layers=3,
12    n_epochs=10,
13 )
14
15 ddpm = DDPM(config)
16
17 n_samples = 256
18
19 data = torch.randn(n_samples, 2, 50)
20
21 conditioning = torch.randn(n_samples, 1, 50)
22
23 dataset = TensorDataset(data, conditioning)
24
25 ddpm.fit(dataset)
26
27 samples = ddpm.generate(conditioning[:8], n_samples=8,)
```

Import the library

Change defaults

Fit the model

Sample

diffusion/

— diffusionsde/

— __init__.py

— config.py

— diffusion.py

— models.py

— sampling.py

— trainer.py

— evaluator.py

— distances.py

— inverse.py

— plotting.py

— logging.py

— notebooks/

— ornstein_uhlenbeck.ipynb

— duffing_generation.ipynb

— tests/

— ...

— .github/workflows/ci.yml

— pyproject.toml

— README.md

```

— diffusionsde/
  |— __init__.py
  |— config.py
  |— diffusion.py
  |— models.py
  |— sampling.py
  |— trainer.py
  |— evaluator.py
  |— distances.py
  |— inverse.py
  |— plotting.py
  |— logging.py
— notebooks/
  |— ornstein_uhlenbeck.ipynb
  |— duffing_generation.ipynb
— tests/
  |— ...
— .github/workflows/ci.yml
— pyproject.toml
— README.md

```

`.freeze()` disables gradients with respect to the U-Net parameters while retaining differentiation with respect to its inputs. This is used in the inverse problem.

`__init__.py`

Exposes useful methods to the DDPM class

`fit, generate, save, load, freeze`

Builds the U-Net with the provided config

`self._build_unet(DiffusionConfig)`

diffusion/

diffusionsde/

__init__.py

config.py

diffusion.py

models.py

sampling.py

trainer.py

evaluator.py

distances.py

inverse.py

plotting.py

logging.py

notebooks/

ornstein_uhlenbeck.ipynb

duffing_generation.ipynb

tests/

• • •

```
.github/workflows/ci.yml
```

pyproject.toml

README.md

config.py

Provides the data class for `DiffusionConfig` with defaults


```

1  """Configuration for diffusion models."""
2
3  from dataclasses import dataclass
4
5  import torch
6
7
8  @dataclass
9  class DiffusionConfig:
10     r"""Configuration for the diffusion model.
11
12     Attributes:
13         n_timesteps: Number of DENOISING timesteps T (algorithmic time for diffusion process)
14         trajectory_length: Length of PHYSICAL trajectory (number of time points in data)
15         beta_start: Starting value of noise schedule  $\beta_0$ 
16         beta_end: Ending value of noise schedule  $\beta_T$ 
17         batch_size: Training batch size
18         learning_rate: Optimiser learning rate
19         n_epochs: Number of training epochs
20         hidden_dim: Hidden dimension for U-Net
21         n_layers: Number of layers in U-Net
22         device: Device for computation
23     """
24
25     n_timesteps: int = 1000 # Denoising algorithm timesteps
26     trajectory_length: int = 250 # Physical trajectory length
27     beta_start: float = 0.0001 # Noise schedule: initial value
28     beta_end: float = 0.02 # Noise schedule: final value
29     batch_size: int = 32
30     learning_rate: float = 1e-4
31     n_epochs: int = 50
32     hidden_dim: int = 128 # Neural network architecture
33     n_layers: int = 4 # Neural network architecture
34     n_conditioning_channels: int = (
35         1 # Number of conditioning channels (1 for scalar, >1 for multi-channel)
36     )
37     n_output_channels: int = 1 # Number of output channels (1 for scalar, >1 for multi-channel)
38     use_position_encoding: bool = False # Enable physical position encoding in U-Net
39     dilation_schedule: list[int] | None = None # Per-level dilation factors (None = no dilation)
40     kernel_size: int = 3 # Convolution kernel size (must be odd)
41     causal: bool = False # Use causal convolutions (outputs depend only on the past)
42     loss_type: str = "mse" # "mse" or "huber"
43     huber_delta: float = 1.0 #  $\delta$  for the Huber/smooth-L1 threshold
44     checkpoint_interval: int = 10 # Epoch interval; zero disables periodic checkpoints
45     checkpoint_dir: str | None = None # Optional directory for periodic checkpoints
46     device: str = (
47         "mps"
48         if torch.backends.mps.is_available()
49         else ("cuda" if torch.cuda.is_available() else "cpu")
50     )

```

diffusion/

— diffusionsde/

— __init__.py

— **config.py**

— **diffusion.py**

— models.py

— sampling.py

— trainer.py

— evaluator.py

— distances.py

— inverse.py

— plotting.py

— logging.py

— notebooks/

— ornstein_uhlenbeck.ipynb

— duffing_generation.ipynb

— tests/

— ...

— .github/workflows/ci.yml

— pyproject.toml

— README.md

`diffusion.py`

Initialises the noising schedule in `__init__()`

Provides the maths for the diffusion model *learning* (not inference) and exposes these methods

`forward_diffusion, compute_loss`

__init__()

```
1 self.betas = torch.linspace(  
2     config.beta_start,  
3     config.beta_end,  
4     config.n_timesteps,  
5 )  
6  
7 self.alphas = 1.0 - self.betas  
8  
9 self.alphas_cumprod = torch.cumprod(  
10     self.alphas,  
11     dim=0,  
12 )  
13  
14 self.alphas = 1.0 - self.betas  
15  
16 self.alphas_cumprod = torch.cumprod(  
17     self.alphas,  
18     dim=0,  
19 )
```

forward_diffusion()

```
1 sqrt_alpha_cumprod = self.sqrt_alphas_cumprod[t]
2 sqrt_one_minus_alpha_cumprod = self.sqrt_one_minus_alphas_cumprod[t]
3
4 x_t = sqrt_alpha_cumprod * x_0 + sqrt_one_minus_alpha_cumprod * noise
```

compute_loss()

```
1 # Random diffusion (algorithmic) times are sampled
2 t = torch.randint(
3     0,
4     self.config.n_timesteps,
5     (batch_size,),
6 )
7
8 # Noise is sampled
9 noise = torch.randn_like(x_0)
10
11 # Data are corrupted to those algorithmic times
12 x_t, _ = self.forward_diffusion(
13     x_0,
14     t,
15     noise,
16 )
17
18 # U-Net predicts the noise
19 predicted_noise = self.model(
20     x_t,
21     t,
22     conditioning,
23 )
24
25 # And minimises  $C = \mathbb{E} [ (\epsilon - \epsilon_{\text{model}})^2 ]$ 
26 F.mse_loss(predicted_noise, noise)
```


diffusion/

— diffusionsde/

— __init__.py

— **config.py**

— diffusion.py

— **models.py**

— sampling.py

— trainer.py

— evaluator.py

— distances.py

— inverse.py

— plotting.py

— logging.py

— notebooks/

— ornstein_uhlenbeck.ipynb

— duffing_generation.ipynb

— tests/

— ...

— .github/workflows/ci.yml

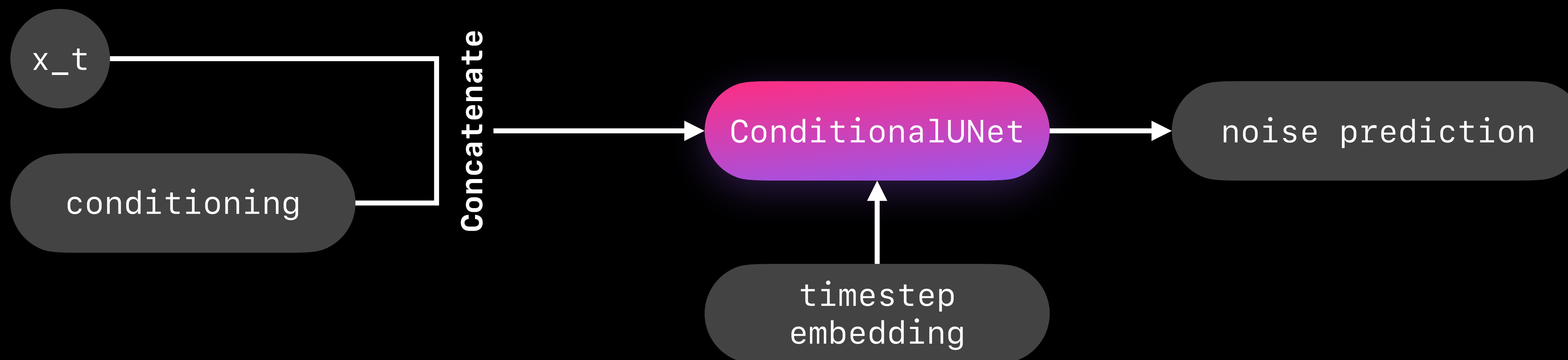
— pyproject.toml

— README.md

models.py

Defines the neural network classes for the U-Net and the
algorithmic time step encoder

SinusoidalPositionalEncoding,
ConditionalUNet



diffusion/

— diffusionsde/

— __init__.py

— **config.py**

— diffusion.py

— models.py

— **sampling.py**

— trainer.py

— evaluator.py

— distances.py

— inverse.py

— plotting.py

— logging.py

— notebooks/

— ornstein_uhlenbeck.ipynb

— duffing_generation.ipynb

— tests/

— ...

— .github/workflows/ci.yml

— pyproject.toml

— README.md

sampling.py

Exposes DDPM (“ancestral” sampling) and DDIM (skip some steps sampling)

DDPMSampler, DDIMSampler

Restriction of autograd graph to a subset of sampled steps for DDIM is defined here

diffusion/

— diffusionsde/

— __init__.py

— **config.py**

— diffusion.py

— models.py

— sampling.py

— **trainer.py**

— evaluator.py

— distances.py

— inverse.py

— plotting.py

— logging.py

— notebooks/

— ornstein_uhlenbeck.ipynb

— duffing_generation.ipynb

— tests/

— ...

— .github/workflows/ci.yml

— pyproject.toml

— README.md

trainer.py

Exposes the training loop: optimiser, learning rate scheduling, metrics tracking and `train()` loop.

Also exposes saving and loading checkpoints.

diffusion/

— diffusionsde/

— __init__.py

— **config.py**

— diffusion.py

— models.py

— sampling.py

— trainer.py

— **evaluator.py**

— distances.py

— inverse.py

— plotting.py

— logging.py

— notebooks/

— ornstein_uhlenbeck.ipynb

— duffing_generation.ipynb

— tests/

— ...

— .github/workflows/ci.yml

— pyproject.toml

— README.md

evaluator.py

Exposes statistics for the generated trajectories:

```
compute_statistics(), compute_metrics(),  
    compute_acf_metrics(),  
    compute_psd_metrics(),  
    compute_distributional_metrics(),
```

diffusion/

— diffusionsde/

— __init__.py

— **config.py**

— diffusion.py

— models.py

— sampling.py

— trainer.py

— evaluator.py

— **distances.py**

— inverse.py

— plotting.py

— logging.py

— notebooks/

— ornstein_uhlenbeck.ipynb

— duffing_generation.ipynb

— tests/

— ...

— .github/workflows/ci.yml

— pyproject.toml

— README.md

distances.py

This should be combined with evaluator.py in the future.

`moment_matching_loss()`

diffusion/

— diffusionsde/

— __init__.py

— **config.py**

— diffusion.py

— models.py

— sampling.py

— trainer.py

— evaluator.py

— distances.py

— **inverse.py**

— plotting.py

— logging.py

— notebooks/

— ornstein_uhlenbeck.ipynb

— duffing_generation.ipynb

— tests/

— ...

— .github/workflows/ci.yml

— pyproject.toml

— README.md

inverse.py

```
1 problem = InverseProblem(  
2     ddpm=model,  
3     observed=data,  
4     build_conditioning=build_conditioning,  
5     parameter_range=(p_min, p_max),  
6 )  
7  
8 result = problem.solve()  
9  
10 result.estimate  
11 result.losses  
12 result.trajectories
```

1. Convert parameter p into form expected by conditioning
2. Generate trajectories with DDIM
3. Compare generated trajectories with observations
4. Compute the moment-matching loss
5. Update p by gradient descent (Adam)

Demo